

# Guide to the Microprocessor Simulator

Adam Greenwood-Byrne

27 April 2026

This simulation allows you to see how a program is executed within the microprocessor of a computer. It will illustrate the way in which machine code instructions are made up and decoded to produce the correct sequence of micro instructions for execution. The program provides a clear picture of the FETCH EXECUTE CYCLE and of the behaviour of complex addressing modes within the instructions.

The simulator supports 32 different instructions and 9 different addressing modes. Instructions can operate on 8 internal registers. This gives a total of 1444 different instructions which can be used.

To run the simulation use CHAIN SIM.

## Contents

|       |                                                                   |    |
|-------|-------------------------------------------------------------------|----|
| 1     | The main screen display                                           | 2  |
| 2     | Main menu options                                                 | 3  |
| 2.1   | Enter data                                                        | 3  |
| 2.2   | Set program counter                                               | 3  |
| 2.3   | Set program counter                                               | 4  |
| 2.4   | Single step by instructions                                       | 4  |
| 2.5   | Single step by micro instructions                                 | 4  |
| 2.5.1 | Manual operation                                                  | 4  |
| 3     | Instruction formats                                               | 4  |
| 3.1   | EXAMPLE 1 - Transfer the contents of register 5 to register 2     | 5  |
| 3.2   | EXAMPLE 2 - Halt program execution                                | 5  |
| 3.3   | EXAMPLE 3 - Load register 3 with the data in memory location 000F | 5  |
| 4     | Addressing modes and assmebly language                            | 6  |
| 4.1   | Absolute addressing mode                                          | 6  |
| 4.2   | Relative addressing mode                                          | 6  |
| 4.3   | Immediate addressing mode                                         | 7  |
| 4.4   | Indexed addressing mode                                           | 7  |
| 4.5   | Register addressing mode                                          | 7  |
| 4.6   | Indirect addressing mode                                          | 8  |
| 4.7   | Index register post increment addressing mode                     | 8  |
| 4.8   | Index register pre decrement addressing mode                      | 8  |
| 4.9   | Implied addressing mode                                           | 8  |
| 5     | APPENDIX                                                          | 8  |
| 5.1   | The simulator instruction set                                     | 9  |
| 5.2   | Addressing modes                                                  | 10 |
| 5.3   | Micro instructions                                                | 11 |

# 1 The main screen display

The upper three quarters of the screen illustrates the computer architecture. This display remains on the screen for all of the program options. The lower quarter of the screen is used to display the current menu. This is initially the MAIN MENU which lists all of the options for running the simulation.

Before using the program it is essential to become familiar with the processor architecture as shown on the screen. This section must be read while viewing the screen display.

Most of the display shows the contents of the CENTRAL PROCESSING UNIT. Lines show data connections and the arrows show the directions of data flow. The right hand side of the display illustrates the IMMEDIATE ACCESS STORE, i.e. the MEMORY of the computer. This consists of a set of 128 locations (with ADDRESSES from 0000 to 007F hexadecimal) each of which contains a 16 bit number (equivalent to 4 hexadecimal digits). All data on the diagram is represented using 4 hexadecimal digits.

The CPU communicates with the MEMORY via an ADDRESS BUS which has 7 lines (or bits) since only 7 lines are necessary to address the 128 memory locations. Data passes between memory and the CPU via a 16 bit DATA BUS. This carries data in both directions. 16 lines are necessary to carry each of the 16 bits of data making up one WORD of the store.

The MEMORY ADDRESS REGISTER (MAR) is used to select a memory location whenever data is to be READ from memory or WRITTEN to memory. Only 22 of the 128 locations can be displayed on the screen. This is done in a window to the memory. Whenever the memory address register points to a location not currently displayed, the locations within the window scroll until the desired location is in the display.

The MEMORY BUFFER REGISTER (MBR) is used as a temporary store for data which is about to be written to memory or which has just been read from memory.

The INTERNAL DATABASE BUS is also 16 bits wide and is used for all data transfers within the CPU. Hence each of the data storage locations (REGISTERS) in the CPU is connected to this data bus.

The CLOCK is not a register but is used to synchronise the operations taking place within the CPU. It produces a square wave with a constant time period. Typically there will be 2 million clock cycles in 1 second (i.e. a 2 MHz clock). In this simulation the clock is reset to zero at the beginning of each instruction so that instructions can be timed in clock cycles.

When an instruction is FETCHED from memory it is loaded into the INSTRUCTION REGISTER (IR). This is actually 32 bits wide. Most instructions use only the leftmost 16 bits (IR) but some instructions need the additional storage provided by the rightmost 16 bits (IR2). This will become apparent when using the simulation.

The instruction in the register (IR) is passed to the CONTROL UNIT where it is decoded. The simulation shows the decoding in the box on the screen. There are 4 parts to this: an instruction code, an addressing mode and two registers (Ra and Rb). All four of these are shown on the screen but the registers Ra and/or Rb may not be required by the instruction being used; in which case they are ignored. The control unit uses the decoded instruction to send the correct sequence of signals to other parts of the CPU to ensure that the instruction is executed correctly. These control lines are omitted from the diagram for clarity.

The PROGRAM COUNTER (PC) is a register which holds the address of the memory location holding the next program instruction or, in the case of a 2 word instruction, it holds the address of the second word of the instruction until this has been fetched from memory.

The STACK POINTER (SP) is a register which holds the address of a location in memory called the top of the stack. The STACK is a set of memory locations used for storing data values, especially return addresses when subroutines are used.

There are 8 general purpose registers labelled R0 to R7. These are the registers referred to by the Ra and Rb values used in the instruction codes.

The ARITHMETIC LOGIC UNIT (ALU) carries out all of the arithmetic and logical operations within the CPU. It has two input registers A1 and A2 which are used for feeding the appropriate data to the ALU. The ALU usually contains a register called the ARITHMETIC REGISTER which holds data and results during the processing stages. The ALU usually also works in conjunction with another register called the ACCUMULATOR which is used for receiving and holding the results of a computation or data transfer. For clarity this simulation has combined the roles of the two registers and has placed the accumulator within the ALU.

Whenever data enters the ALU via the A1 input buffer, this is immediately displayed as 16 binary digits in the ACCUMULATOR. Any arithmetic or logical operation produces a result which is also displayed in the accumulator. This is especially useful for viewing shifts and rotations and for seeing why the status flags are changing.

Results of operations in the ALU affect the status of 3 flags. Two of these N (negative) and Z (zero) are in the STATUS REGISTER. The third C (carry) would also be in the status register but because of its special role in shifts and rotations it has been placed near the accumulator in the ALU to make it easier to see how these operations work.

The Carry flag is set to 1 if the value in the accumulator has overflowed (i.e. is too big to store).

The Zero flag is set to 1 if the value in the accumulator is zero.

The Negative flag is set to 1 if the value in the accumulator is negative. Since two's complement numbers are assumed here this means that the most significant bit in the accumulator must be a 1 to signify a negative number. Thus the N flag simply takes on the value of bit 15 of the accumulator.

Note that none of the flags is set simply by transferring data to A1. The flags are set only as a result of processes carried out within the ALU.

## 2 Main menu options

### 2.1 Enter data

Before any of the options can be used effectively it is necessary to enter a program into the memory. (See later for details of the instructions formats.) The sub menu for data entry provides a number of useful features.

Since most programs will be entered starting from location zero there is a simple means of jumping to location zero (press Z).

Data is entered in sets of 4 hexadecimal digits. Pressing RETURN afterwards is not necessary. Since the majority of data values are entered sequentially the data pointer automatically moves to the next location after a data entry.

To move to another location for data entry simply use the up and down cursor keys.

Simple program editing facilities have been provided. Pressing I inserts a blank location BEFORE the current location by moving all of the subsequent data down one location. Similarly, pressing R removes the data in the present location by moving all subsequent data values up one location in memory. It is important to realise that any JUMP (i.e. JMP or JSR) instructions will need to have their addresses changed if this type of editing is used without some thought for the consequences.

When data entry is complete, pressing Q returns to the main menu.

### 2.2 Set program counter

When there is a program in memory the next step is to set the program counter to point to the first instruction in the program (or to some other instruction if execution

is to start part way through the program). The address of the first instruction is entered as 4 hexadecimal digits. You are now returned to the main menu.

## 2.3 Set program counter

This option will run the program starting at the location specified by the program counter. As the program runs the display will show all of the data transfers which take place. It will also show the decoded instructions in the control unit and the sequence of micro instructions necessary to execute the command. These are displayed in a window at the bottom of the screen.

The run option is not really intended to provide a clear picture of what is happening. Rather, it is a means of fairly rapid program execution when the program is known to be working effectively. There is however an option for slow running. This waits for 1 second after each data transfer and the data which has moved is made to flash so that attention is drawn to it. Even so, it is difficult to follow program execution in detail using this mode.

Pressing Q (Quit) will return you to the main menu when the current instruction has been executed.

## 2.4 Single step by instructions

This option provides the same fast and slow execution speeds as the previous option except that it pauses after each instruction has been completed. Pressing the SPACE-BAR will result in the next instruction being executed.

This option is ideal for checking that each instruction has worked as intended. It is not, however, designed to show how the instruction is execution. Micro instructions are displayed in the lower window but these may well go past too quickly for a complete analysis.

## 2.5 Single step by micro instructions

Here the program pauses after each micro instruction. The micro instruction is displayed in the lower window and its effect can be seen in the main display. Pressing the SPACE-BAR will result in the next micro instruction being carried out. The FETCH and EXECUTE phases of the FETCH EXECUTE CYCLE are clearly displayed alongside this window.

The purpose of this option is to give a clear understanding of the sequence of operations needed to carry out a variety of different instructions. It is also invaluable as a means of discovering why a particular instruction (or series of instructions) does not work as expected.

The Quit option (Q) will return you to the main menu on completion of the current instruction. Use "Q" to single step through the instruction until it is complete.

### 2.5.1 Manual operation

This option is provided as a means of testing your understanding of the sequences of micro instructions necessary to carry out each instruction. With a program loaded into memory and the program counter set correctly, you must enter the micro instructions one by one to carry out the necessary steps in the fetching and execution of instructions. (See page 14 for a complete list of micro instructions.)

This option is of little value until a good understanding of the simulator has already been achieved.

## 3 Instruction formats

Each instruction is 16 bits long. It is made up as follows:

|                 |        |
|-----------------|--------|
| Operation code  | 5 bits |
| Addressing mode | 3 bits |
| Register Ra     | 4 bits |
| Register Rb     | 4 bits |

A complete list of operation codes and addressing mode codes is given later. All 32 combinations of 5 bits have been used to provide 32 different operation codes. These are given in the instruction set. All 8 combinations of 3 bits have been used for 8 different addressing modes together with a ninth addressing mode (implied) which is implied by the operation code and thus needs no special code of its own. Whatever combination of 3 bits is used this will be ignored.

### 3.1 EXAMPLE 1 - Transfer the contents of register 5 to register 2

Find the correct instruction from the list of instructions. This is the MOV (move) instruction. It requires the register (REG) address mode. In this mode Ra is the source register and Rb the target register. The instruction has no second word.

|                      |       |
|----------------------|-------|
| Operation code (MOV) | 10000 |
| Register mode        | 100   |
| Ra = 5               | 0101  |
| Rb = 2               | 0010  |

The complete instruction is 10000 100 0101 0010

Next partition this into four groups of 4 bits each

i.e. 1000 0100 0101 0010

Each group converts to one hexadecimal digit producing the hexadecimal instruction code 8452

### 3.2 EXAMPLE 2 - Halt program execution

This is the STP instruction. It uses the implied addressing mode which means that the address mode, Ra and Rb are ignored.

|                      |       |
|----------------------|-------|
| Operation code (STP) | 11111 |
| Address mode         | ***   |
| Ra = ?               | ****  |
| Rb = ?               | ****  |

Here the "\*" is used to signify a bit which must be present in the instruction but which can be either 0 or 1. In this example we will replace each \* by 0.

This produces the instruction code 1111 1000 0000 0000 i.e. F800

### 3.3 EXAMPLE 3 - Load register 3 with the data in memory location 000F

This requires the LDR instruction. Here there is a choice of 7 different addressing modes. However, since we are specifying the address of the data this means that the ABSOLUTE addressing mode must be used. (Check the table to see that this is the case) This addressing mode uses a 2 word instruction. The second word is simply the specific address. Ra is ignored in this instruction but Rb is the target register for the load.

|                      |                    |
|----------------------|--------------------|
| Operation code (LDR) | 00001              |
| Absolute mode        | 000                |
| Ra = ?               | ****               |
| Rb = 3               | 0011               |
| Data address         | 000F (hexadecimal) |

This gives the instruction code 0803 000F (2 words, hexadecimal)

## 4 Addressing modes and assembly language

This section will look at examples of the use of the 9 different addressing modes and also at a way of writing the instructions in a human readable format (as opposed to hexadecimal). This notation is referred to as assembly language because instructions written in this form can be translated by an assembler. Here we will be using it only as a readable notation for designing programs. The assembly language instructions will have to be assembled by hand but this is a simple task and will aid understanding of the processes carried out in the CPU.

Instructions for the simulator can be one or two words long. Only the first word is correctly referred to as the INSTRUCTION. The second word (if any) is referred to as the OPERAND. This can be data, an address or an offset used to compute an address. The control unit decodes the INSTRUCTION but does not use the OPERAND. The sequence of operations generated by decoding the instruction ensures that the operand is used in the appropriate way.

### 4.1 Absolute addressing mode

This uses 2 word instructions where the second word represents the address of the data to be used or, in the case of a store (STO) instruction, the second word represents the address of the location in which the data will be stored.

Example 3 on page 6 uses the absolute mode to load register 3 from memory location 000F.

The control unit will decode the instruction (0803) as

```
LDR ABS R0 R3
```

Ra is decoded (R0 here) even though it is not used by the instruction. For this reason it should be obvious that 0873 which decodes as LDR ABS R7 R3 would achieve the same result and is thus effectively the same instruction.

In assembly language the instruction would be written as

```
LDR 000FH,R3
```

Notice that this includes the operand. The "H" stands for hexadecimal and is used to indicate that the 4 digit number preceding it is in this form.

The assembly language instruction

```
STO 006BH,R5
```

means store the contents of register 5 in memory location 006B.

Check that the instruction codes to 0005 006B.

### 4.2 Relative addressing mode

This is used by the set of branch instructions. These allow program execution to branch to a new location conditional on the state of one of the status flags. There is also one unconditional branch instruction.

Register Ra and Rb are not used by these instructions. If the branch is taken the second word of the instruction is used to provide an offset. This is added to the contents of the program counter to produce the address of the next instruction. It is important to remember when calculating these offsets that the program counter will be pointing to the instruction after the branch when the offset is applied. Since offsets can be both positive or negative these are given in twos complement form.

The following example considers the instruction to branch backwards by 6 locations if the last result in the accumulator was negative.

The assembly language is BMI (-6) i.e. branch if minus by -6 locations. More usually this would be written BMI "label" where label is an identifying name which has already been used to identify the line to which the program must branch. Assemblers can

usually deal with labels which makes manual calculation of the offset unnecessary. However, the exercise is important so we will calculate the offsets manually.

Use the instruction set to verify that this instruction should be coded as B900 FFFA. (You should have been able to deduce that the 16 bit twos complement value for -6 is FFFA).

Use the instruction set to confirm that an unconditional branch forwards by ten locations i.e. BRA (10) is given by D100 000A.

### 4.3 Immediate addressing mode

Here the operand is a data value and not an address. It has to be a 2 word instruction since an operand is always expected.

Consider the instruction to add 4 on to the contents of register R6. The assembly language instruction for this is

```
ADD #0004H,R6
```

Notice that the number 0004 is preceded by a #. This signifies that it is a data value and not an address. Without the # this becomes an absolute addressing instruction.

Verify that it codes to 2206 0004

### 4.4 Indexed addressing mode

Here register Ra is used as an index register. It is used in conjunction with the operand which represents an offset (which can be either positive or negative). The offset is added to the index to provide the address of the data to be used. The contents of the index register remain unchanged during this process.

Consider the following instruction:

```
LDR R3,#000CH,R5
```

R3 is the index register and R5 the target register. The instruction is to load register R5 from memory but we can not determine the location unless the contents of the index register (R3 in this case) are known.

Verify that the instruction codes to 0B35 000C

Suppose that R3 contains the value 0034H

The address of the data is found by adding this value to the offset 000CH.

$$000C + 0034 = 0040H$$

Thus the data from location 0040H is transferred to register R5.

### 4.5 Register addressing mode

This uses one word instructions. An example is the move instruction which is illustrated by example 1 on page 6. This would be written as MOV R5,R2 in assembly language.

An arithmetic shift left is used to multiply the contents of a register by 2 (there could be problems with overflow). The following command will multiply the number in register R0 by 2 and put the result in register R1:

```
ASL R0,R1
```

This codes to 6401

It is often useful to specify the same register as source and target. For example, to decrement register R3 use the instruction:

```
DEC R3 R3
```

which codes to 6C33

## 4.6 Indirect addressing mode

Here the indirect register holds the address of the data. In assembly language the indirect register is put in brackets. The following instruction is to store the contents of register R3 in the memory location specified by the address in register R4.

```
STO (R4),R3
```

Verify that this codes to 0543

## 4.7 Index register post increment addressing mode

This mode is very useful for dealing with elements of an array taken sequentially. The contents of the index register are used as the address of the data in exactly the same way as in indirect addressing above. The difference lies in the fact that the index register is incremented immediately after the data transfer. This leaves the index register conveniently pointing at the next element of the array.

There are a number of ways of writing this in assembly language. The following is one possibility:

```
LDR (R2)+,R5
```

R2 is the index register. The + after it indicates that it is incremented after the data transfer (from memory to register R5) has taken place.

Verify that this instruction codes to 0E25

## 4.8 Index register pre decrement addressing mode

This is very similar to the previous mode and is also used for dealing with arrays. The index register again holds the address of the data. This time the contents of the index register are decremented and this is done before the data transfer takes place. Compare the following assembly language instruction with the previous one and make sure that you understand the reason for the differences.

```
LDR -(R2),R5
```

Verify that this codes to 0F25

## 4.9 Implied addressing mode

An example of this was given on page 6, i.e. the command to halt program execution. The assembly language for this is STP. No more is needed because it is an implied instruction.

# 5 APPENDIX

Page 12 contains a complete list of the 32 instructions available for the simulator. The table is a useful summary of the codes, the actions performed by the instruction and the flags that are changed as a result of the instructions being executed.

Page 13 contains a summary of the different addressing modes. It should be used in conjunction with the table of instruction codes for coding instructions prior to entering a program into memory. For details of what the addressing modes do it is necessary to refer back to pages 7-10.

Page 14 contains a complete list of the 42 different micro instructions used by the simulator. There is a brief description of the action performed by each of these. It will be essential to refer to this list when using the manual operation option for running a program.

It is possible to use the micro instructions without first entering a program into memory. However, this is rather limiting because the registers used by the micro instructions are determined by the control unit after decoding an instruction.



## 5.1 The simulator instruction set

| OPC | CODE  | MODE | FLG   | ACTION PERFORMED                             |
|-----|-------|------|-------|----------------------------------------------|
| ADC | 00101 | 7    | CNZ   | Add data (and C) to Rb, result in Rb         |
| ADD | 00100 | 7    | CNZ   | Add data to Rb, result in Rb                 |
| AND | 01001 | 7    | NZ    | Form (data AND Rb), result in Rb             |
| ASL | 01100 | REG  | CNZ   | Arithmetic shift left Ra, result in Rb       |
| BCC | 10100 | REL  |       | Branch if carry clear (C=0) to PC +/- offset |
| BCS | 10101 | REL  |       | Branch if carry set (C=1) to PC +/- offset   |
| BEQ | 10110 | REL  |       | Branch if equals zero (Z=1) to PC +/- offset |
| BMI | 10111 | REL  |       | Branch if minus (N=1) to PC +/- offset       |
| BNE | 11000 | REL  |       | Branch if not zero (Z=0) to PC +/- offset    |
| BPL | 11001 | REL  |       | Branch if plus (N=0) to PC +/- offset        |
| BRA | 11010 | REL  |       | Branch unconditionally to PC +/- offset      |
| CLC | 11100 | IMP  | C     | Clear carry to 0                             |
| CMP | 01000 | 7    | CNZ * | Compare data with Rb, set flags accordingly  |
| DEC | 01101 | REG  | CNZ   | Decrement Ra, result in Rb                   |
| EOR | 01010 | 7    | NZ    | Form (data EOR Rb), result in Rb             |
| INC | 01110 | REG  | CNZ   | Increment Ra, result in Rb                   |
| JMP | 00010 | ABS  |       | Transfer program control to new location     |
| JSR | 00011 | ABS  |       | Jump to subroutine at new location           |
| LDR | 00001 | 7    |       | Load data into register Rb                   |
| LSR | 01111 | REG  | CNZ   | Logical shift right Ra, result in Rb         |
| MOV | 10000 | REG  |       | Transfer data from Ra to Rb                  |
| NOP | 11011 | IMP  |       | No operation, delay of 3 clock cycles        |
| OR  | 01011 | 7    | NZ    | Form (data OR Rb), result in Rb              |
| POP | 10010 | REG  |       | Move data from top of stack to Rb register   |
| PSH | 10011 | REG  |       | Move data from Ra register to top of stack   |
| ROL | 10001 | REG  | CNZ   | Rotate left Ra (with carry), result in Rb    |
| RTS | 11110 | IMP  |       | Return from subroutine (to address on stack) |
| SBC | 00111 | 7    | CNZ   | Calculate Rb - data (inc. C), result in Rb   |
| SEC | 11101 | IMP  | C     | Set carry to 1                               |
| STO | 00000 | 5    |       | Store the contents of Rb in memory           |
| STP | 11111 | IMP  |       | Halt program execution                       |
| SUB | 00110 | 7    | CNZ   | Calculate Rb - data, result in Rb            |

\* Flags are set for the CMP instruction as follows:

Data = Rb      Z=1  
Data < Rb      N=1  
Data > Rb      Z=0 AND N=0

OPC      This is the instruction code mnemonic

CODE      This is the 5 bit binary code for the instruction.

MODE      This gives the allowed addressing mode(s) for the instruction:  
7 means ABS IMM IDX REG IND RPI RPD  
5 means ABS IDX IND RPI RPD

FLG      This gives the flags which are affected by the instruction:  
C = carry      N=negative      Z = zero

## 5.2 Addressing modes

| MODE                          | MNE | CODE | Ra                | Rb              | Contents of next location |
|-------------------------------|-----|------|-------------------|-----------------|---------------------------|
| ABSOLUTE                      | ABS | 000  | ignored           | active register | effective address         |
| RELATIVE                      | REL | 001  | ignored           | ignored         | offset +/-                |
| IMMEDIATE                     | IMM | 010  | ignored           | target register | data                      |
| INDEXED                       | IDX | 011  | index register    | target register | offset +/-                |
| REGISTER                      | REG | 100  | source register   | target register | none                      |
| INDIRECT                      | IND | 101  | indirect register | target register | none                      |
| INDEX REGISTER POST INCREMENT | RPI | 110  | index register    | target register | none                      |
| INDEX REGISTER POST DECREMENT | RPD | 111  | index register    | target register | none                      |
| IMPLIED                       | IMP | ***  | ignored           | ignored         | none                      |

\*\*\*      The code for IMPLIED mode can be any combination of 3 bits

MNE      This is the 3 letter mnemonic for the addressing mode

CODE      This is the 3 bit code for the addressing mode

Where the entry for Ra or Rb is ignored any combination of 4 bits can be used to make up the code.

Where the entry for contents of next location is none the instruction consists of one word rather than two.

### 5.3 Micro instructions

|      |     |                                                                     |
|------|-----|---------------------------------------------------------------------|
| ADC  | REG | Calculate $A1 + A2 + C$ , put result in register, update C,N,Z      |
| ADD  | MAR | Calculate $A1 + A2$ , result in memory register, update C,N,Z       |
| ADD  | PC  | Calculate $A1 + A2$ , result in program counter, update C,N,Z       |
| ADD  | REG | Calculate $A1 + A2$ , put result in register, update C,N,Z          |
| AND  | REG | Calculate $A1 \text{ AND } A2$ , put result in register, update N,Z |
| ASL  | REG | Arithmetic shift left accumulator, result in register, (CNZ)        |
| CLC  | ALU | Clear carry flag to zero                                            |
| CMP  | ACC | Calculate $A1 - A2$ and update C,N,Z                                |
| DEC  | REG | Subtract 1 from A1, put result in register, update C,N,Z            |
| DEC  | SP  | Decrement (subtract 1 from) stack point                             |
| EOR  | REG | Calculate $A1 \text{ EOR } A2$ , put result in register, update N,Z |
| INC  | PC  | Increment (add 1 to) program counter                                |
| INC  | REG | Add 1 to A1, put result in register, update C,N,Z                   |
| INC  | SP  | Add 1 to stack pointer                                              |
| IR2  | PC  | Move data from instruction register 2 to program counter            |
| LSR  | REG | Logical shift left accumulator, result in register, (CNZ)           |
| MBR  | A1  | Move data from memory buffer register to ALU input A1               |
| MBR  | A2  | Move data from memory buffer register to ALU input A2               |
| MBR  | IR  | Move data from memory buffer to instruction register                |
| MBR  | IR2 | Move data from memory buffer to instruction register 2              |
| MBR  | MAR | Move data from memory buffer to memory address register             |
| MBR  | PC  | Move data from memory buffer register to program counter            |
| MBR  | REG | Move data from memory buffer to register                            |
| MBR  | SP  | Move data from memory buffer register to stack pointer              |
| OR   | REG | Calculate $A1 \text{ OR } A2$ , put result in register, update N,Z  |
| PC   | A2  | Move data from program counter to ALU input A2                      |
| PC   | MAR | Move data from program counter to memory address register           |
| PC   | MBR | Move data from program counter to memory buffer register            |
| READ | MEM | Move data from memory location to memory buffer register            |
| REG  | A1  | Move data from register to ALU input A1                             |
| REG  | A2  | Move data from register to ALU input A2                             |
| REG  | MAR | Move data from register to memory address register                  |
| REG  | MBR | Move data from register to memory buffer register                   |
| REG  | SP  | Move data from register to stack pointer                            |
| ROL  | REG | Rotate left accumulator and C, result in register, (C,N,Z)          |
| SBC  | REG | Calculate $A2 - A1$ (with borrow), result in register, (C,N,Z)      |
| SEC  | ALU | Set carry to 1                                                      |
| SP   | MAR | Move data from stack pointer to memory address register             |
| SP   | MBR | Move data from stack pointer to memory buffer register              |
| SP   | REG | Move data from stack pointer to register                            |
| SUB  | REG | Calculate $A2 - A1$ , result in register, updated C,N,Z             |
| WRI  | MEM | Move data from memory buffer register to memory location            |

Each of the instructions in the instruction set of the microprocessor must be broken down into micro instructions before execution. The table above gives the full set of micro instructions for the simulator.

The sequence of micro instructions which is executed for any given instruction is determined by the control unit when the instruction is decoded. This also determines which of the two registers (Ra or Rb) is referred to by REG in the micro instruction.